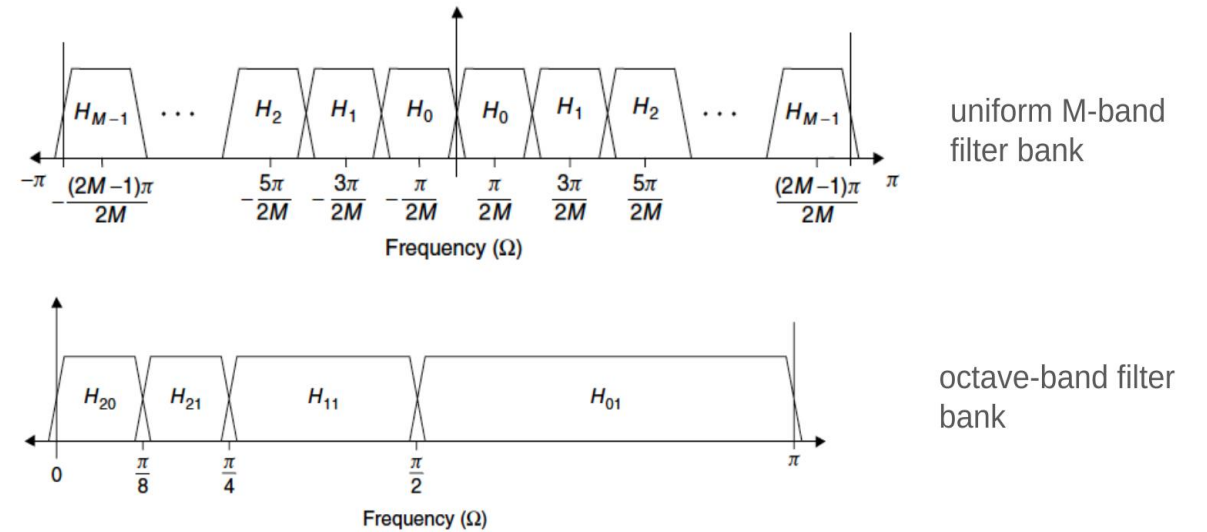


3. Time-frequency representations

Filter banks and transforms

From filters to compact time-frequency representations for audio coding.

Magnitude frequency responses of filter banks



Learning goals

- Understand FIR/IIR filters through their frequency response.
- Explain downsampling, upsampling and anti-aliasing/interpolation filters.
- Describe analysis-synthesis filter banks and critical sampling.
- Relate DFT, DCT, STFT and MDCT to audio coding.
- Discuss time-frequency tradeoffs and transient handling.

Finite Impulse Response filters

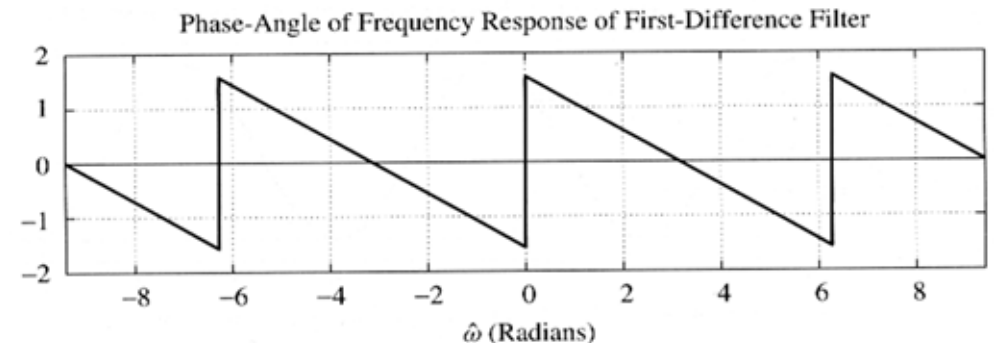
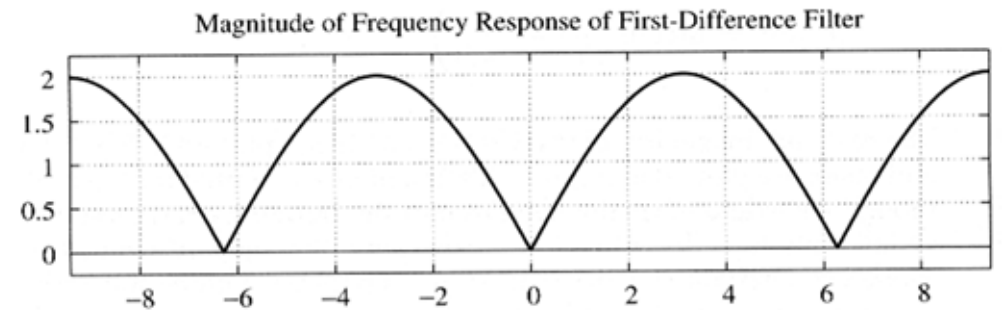
$$y[n] = x[n] - x[n-1] \quad H(\omega) = 1 - e^{-j\omega} = 1 - \cos\omega + j \sin\omega$$

$$|H(\omega)| = [(1 - \cos\omega)^2 + \sin^2\omega]^{1/2} = [2(1 - \cos\omega)]^{1/2} = 2|\sin(\omega/2)|$$

$$\angle H(\omega) = \arctan(\sin\omega / (1 - \cos\omega))$$

$$y[n] = \sum_{k=0}^M b_k x[n - k] = \sum_{k=0}^M h[k] x[n - k]$$

$$H(\omega) = \sum_{k=0}^M b_k e^{-j\omega k} = \sum_{k=0}^M h[k] e^{-j\omega k}$$



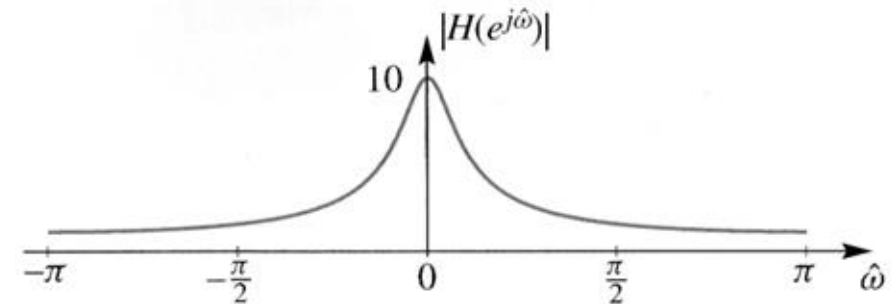
Infinite Impulse Response filters

$$y[n] = \sum_{l=1}^N a_l y[n-l] + \sum_{k=0}^M b_k x[n-k]$$

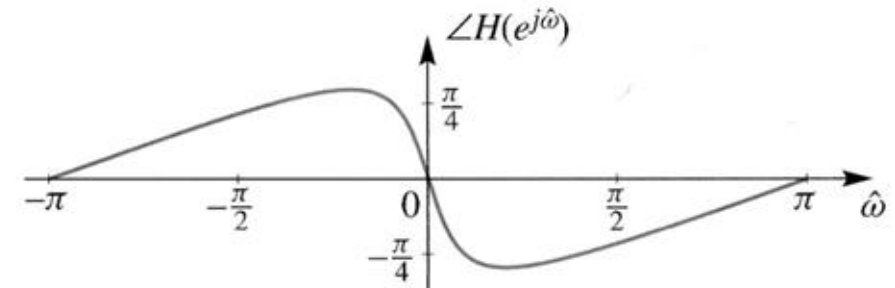
$$y[n] = h[n] * x[n] \Leftrightarrow Y(z) = H(z)X(z)$$

$$H(\omega) = H(e^{j\omega}) = H(z)|_{z=e^{j\omega}}$$

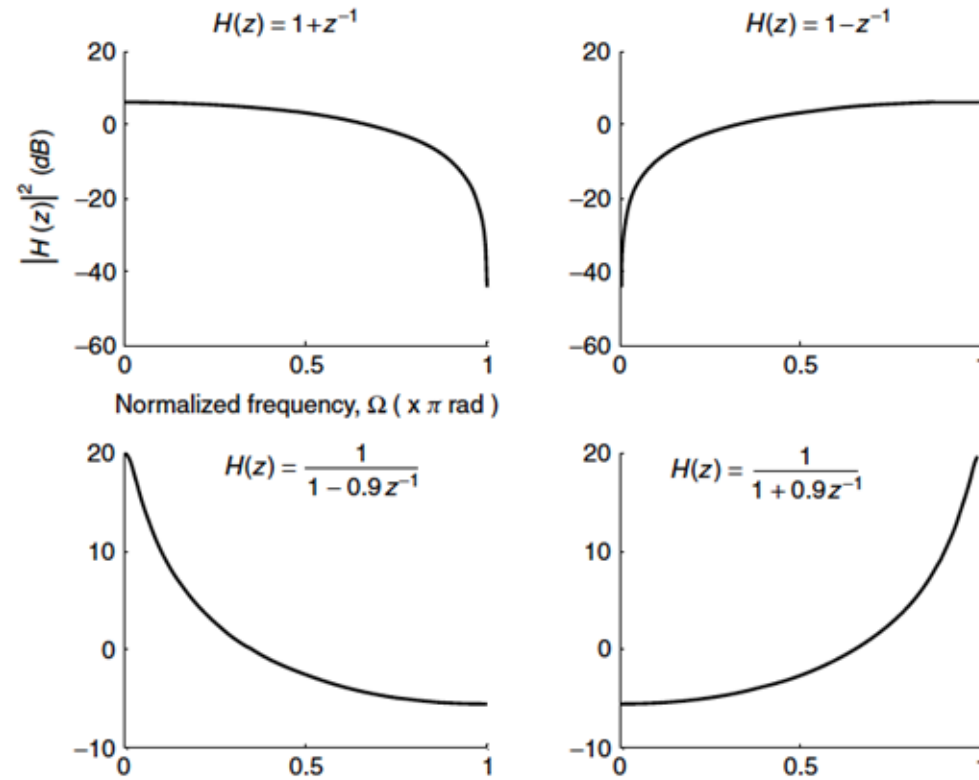
$$H(z) = 1 / (1 - 0.8z^{-1})$$



(a)



FIR and IIR examples



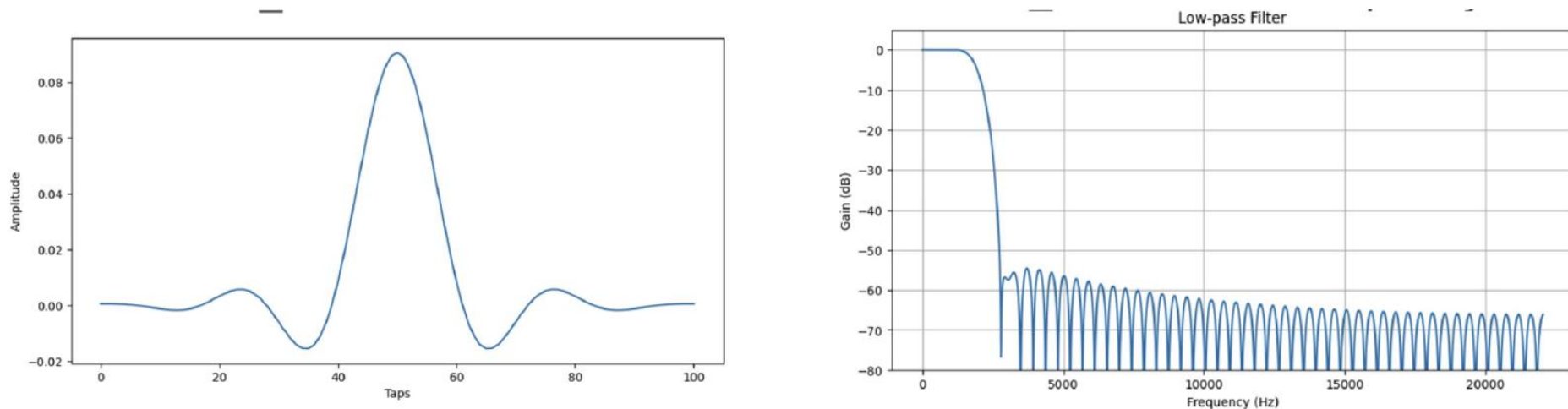
Filter design: window method (`firwin`)

Windowing a theoretically ideal impulse response $h(n)$ by some suitable chosen window function $w(n)$, yielding $h_w(n) = w(n) \cdot h(n)$, $n \in \mathbb{Z}$.

Ex: the ideal impulse response of a lowpass filter is the sinc function

$$h(n) = B \cdot \text{sinc}(Bn) \triangleq B \cdot \sin(\pi Bn)/(\pi Bn), \quad n \in \mathbb{Z},$$

where $B = 2f_c$ is the normalized bandwidth in Hz and f_c is the cutoff frequency in Hz.



filter length = 101, $f_c = 2000$ Hz

Downsampling

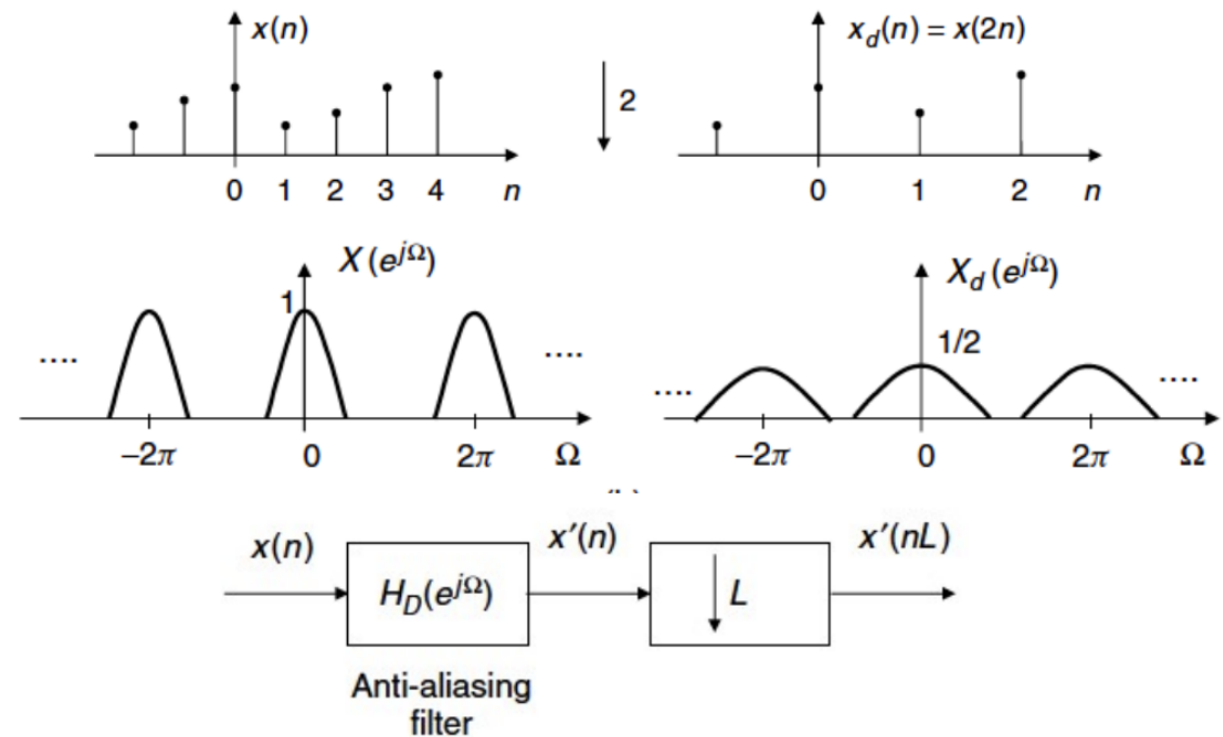
$$x_d(n) = x(nL)$$

Keep every L-th sample.
The sampling rate becomes f_s/L .

Low-pass filter first, then discard samples.

Remove frequencies above the new Nyquist limit $f_s/(2L)$ to prevent aliasing.

Example: $48 \text{ kHz} / 2 = 24 \text{ kHz}$.
New Nyquist limit: 12 kHz .



Upsampling

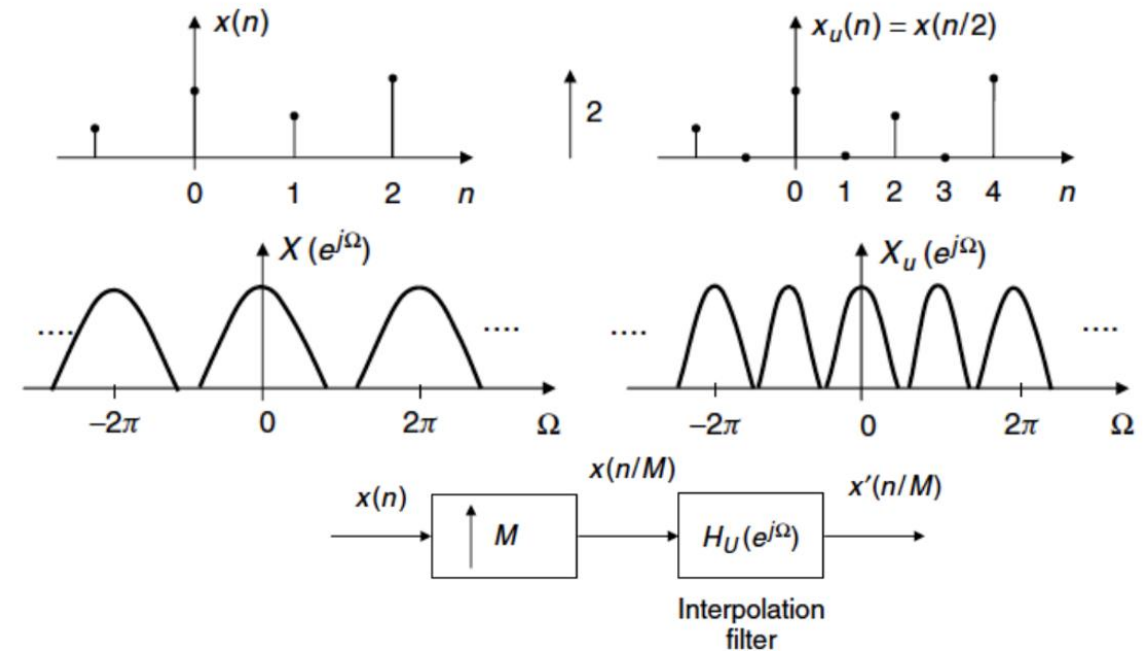
$$x_u(n) = \sum_{m=-\infty}^{\infty} x(m)\delta(n - mM) = \begin{cases} x(n/M), & n = 0, \pm M, \pm 2M, \dots \\ 0, & \text{otherwise} \end{cases}$$

Insert $M-1$ zeros between samples.
The sampling rate becomes $M \cdot f_s$.

Zero insertion creates spectral images:
copies of the original spectrum.

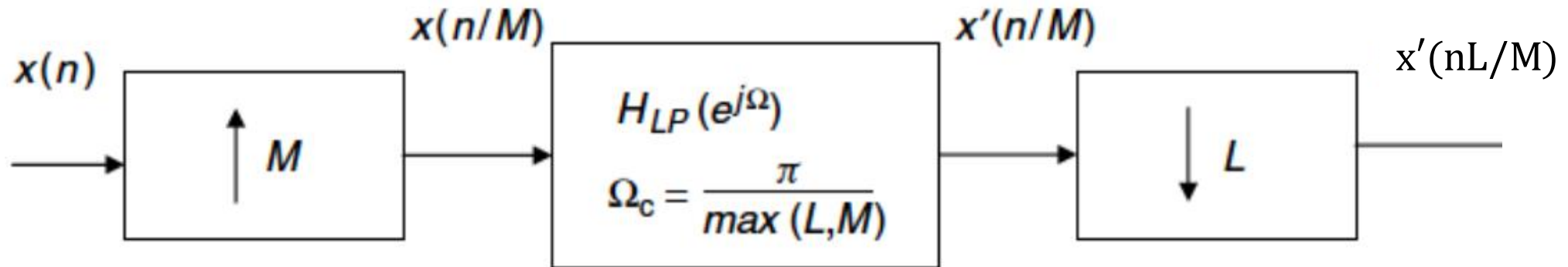
An interpolation low-pass filter
removes the images and fills the new
samples.

Example: $24 \text{ kHz} \times 2 = 48 \text{ kHz}$.
The original bandwidth stays the same.



Sampling rate changes by non-integer factor

Use a rational factor M/L . Upsample by M , low-pass filter, then downsample by L .



Output sampling rate = input sampling rate $\times M/L$

Example: 44.1 kHz to 48 kHz uses $M/L = 160/147$.

One filter suppresses both images and aliasing. Its bandwidth is limited by the lower of the two Nyquist frequencies.

Filter banks

An input signal $s(n)$ is processed at the encoder by a parallel bank of $(L - 1)$ -th order FIR bandpass filters, $H_k(z)$. The bandpass analysis outputs,

$$v_k(n) = h_k(n) * s(n) = \sum_{m=0}^{L-1} s(n-m)h_k(m), \quad k = 0, 1, \dots, M-1$$

are decimated by a factor of M , yielding the subband sequences

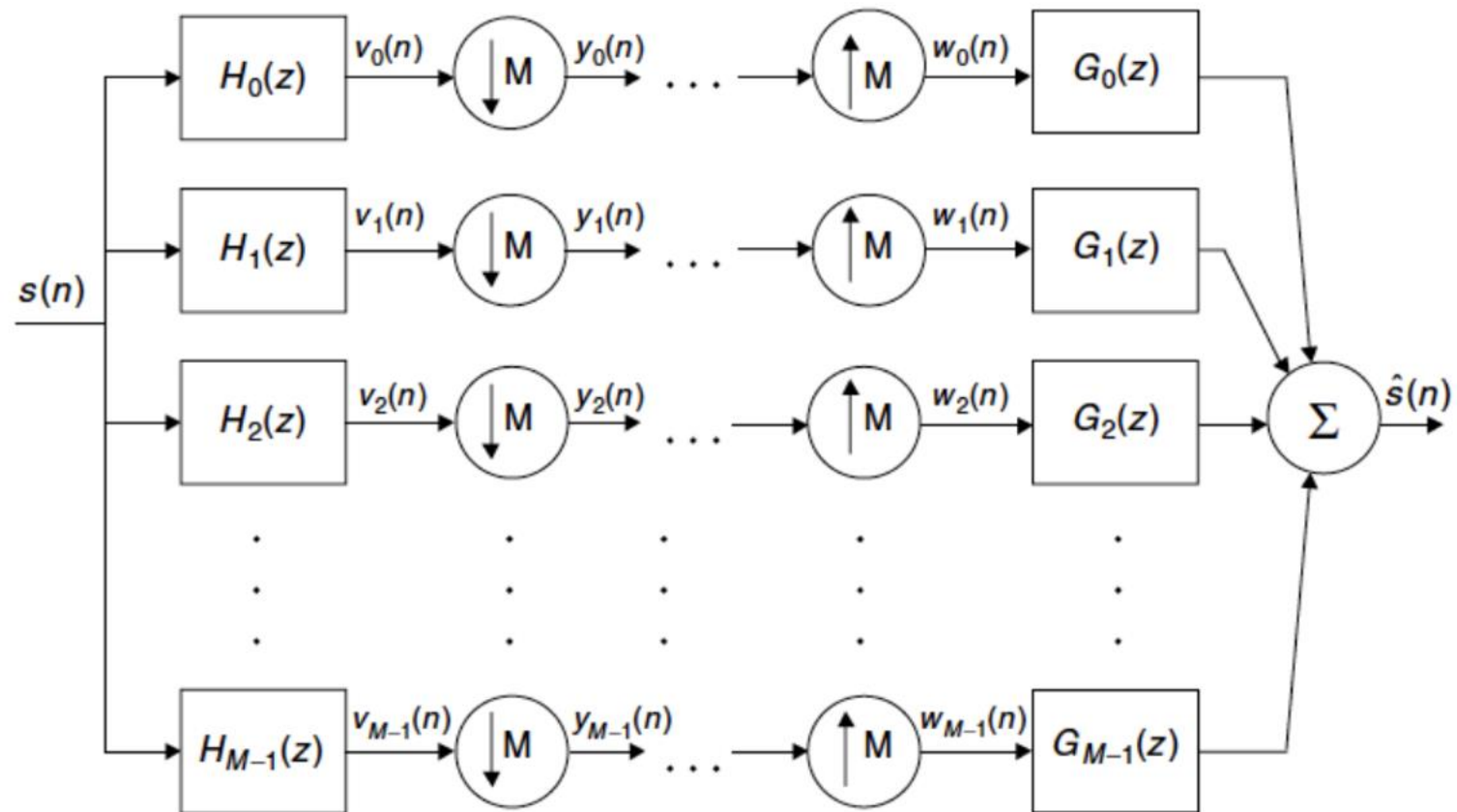
$$y_k(n) = v_k(Mn) = \sum_{m=0}^{L-1} s(nM-m)h_k(m), \quad k = 0, 1, \dots, M-1$$

which comprise a critically sampled or maximally decimated signal representation, i.e., the number of subband samples is equal to the number of input samples.

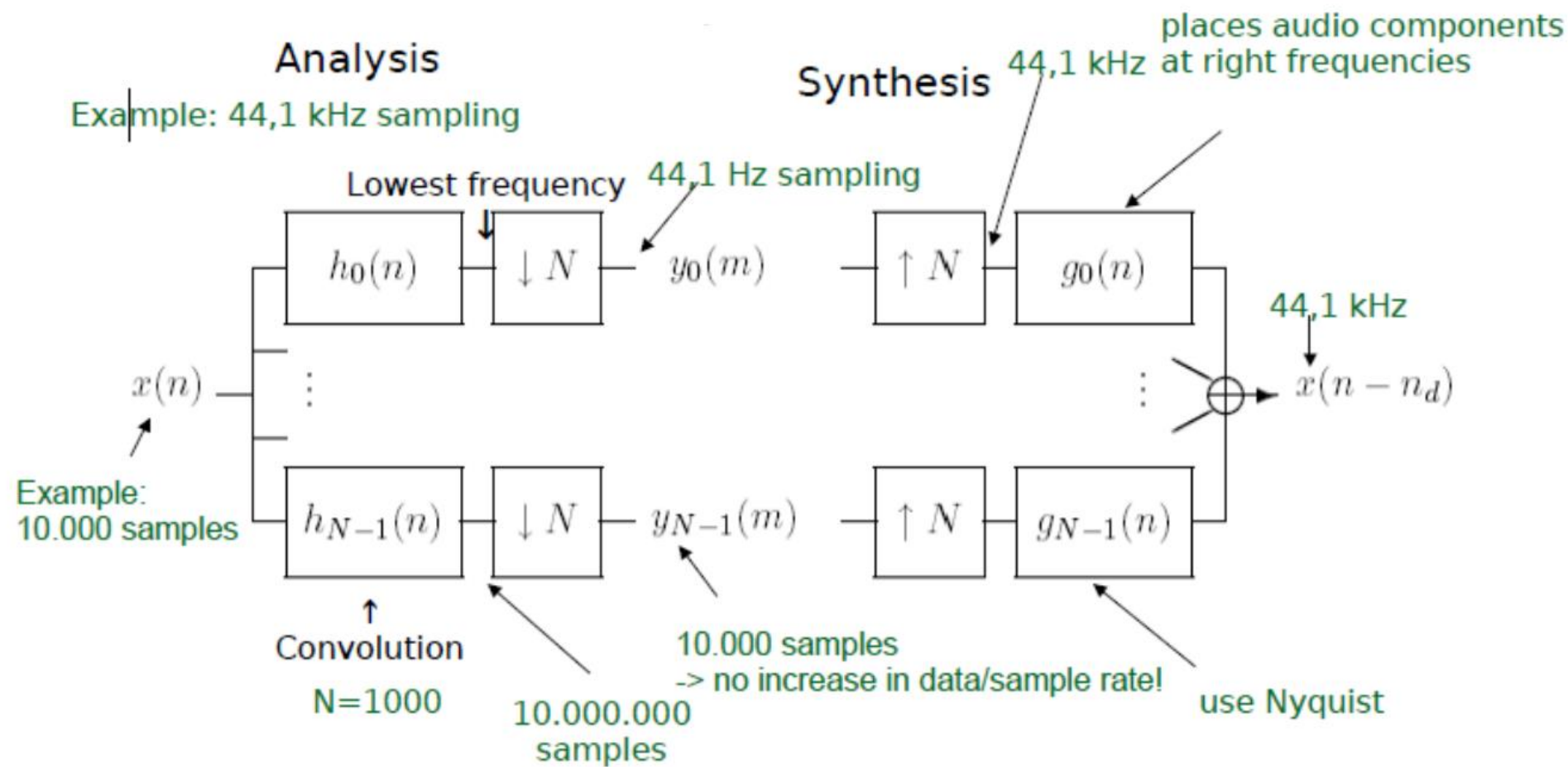
- Because it is impossible to achieve perfect “brickwall” magnitude responses with finite-order bandpass filters, there is unavoidable aliasing between the decimated subband sequences.
- Quantization and coding are performed on the subband sequences $y_k(n)$.
- The quantized subband samples are eventually received by the decoder, where they are upsampled by M .
- To eliminate the imaging distortions introduced by the upsampling the samples are processed by a parallel bank of synthesis filters, $G_k(z)$, and the filter outputs are combined to form the overall output.

$$\hat{s}(n) = \frac{1}{M} \sum_{m=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} \sum_{k=0}^{M-1} s(m) h_k(lM - m) g_k(l - Mn)$$

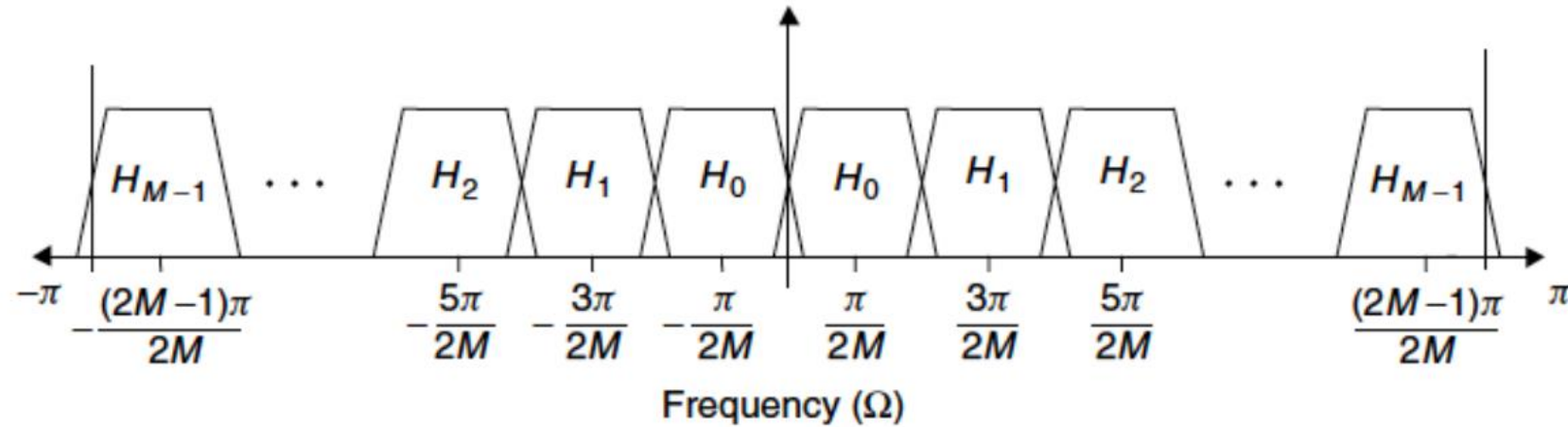
$$\hat{S}(\Omega) = \frac{1}{M} \sum_{k=0}^{M-1} \sum_{l=0}^{M-1} S\left(\Omega + \frac{2\pi l}{M}\right) H_k\left(\Omega + \frac{2\pi l}{M}\right) G_k(\Omega)$$



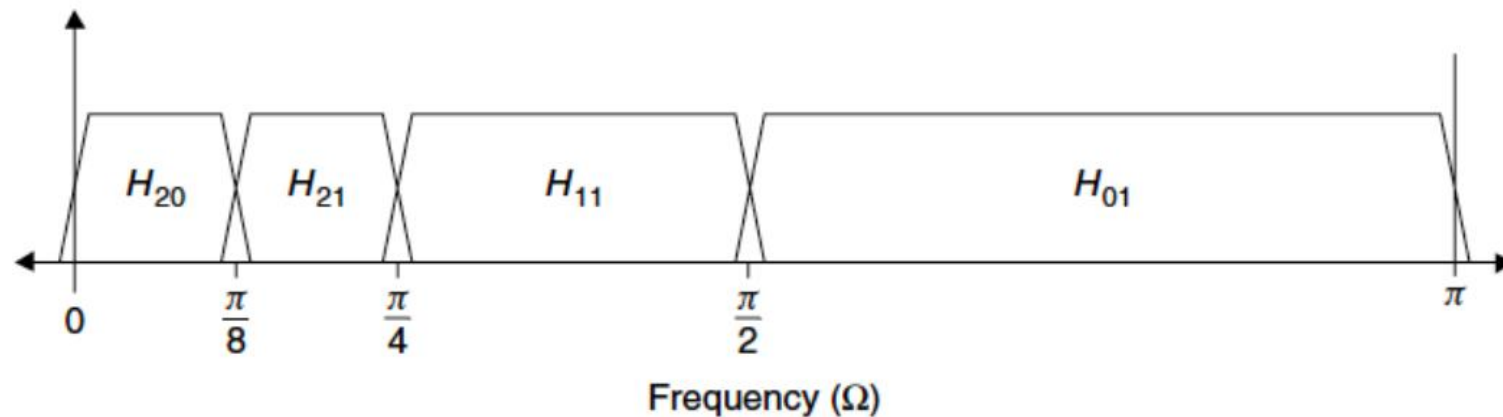
Uniform M-band maximally decimated analysis-synthesis filter bank.



Magnitude frequency responses of filter banks



uniform M-band
filter bank



octave-band filter
bank

Ideal filters

- Ideal filters are not realizable.
- In the time domain they would mean a convolution of our signal with a Sinc function.
- Sinc function is infinitely long and not causal, meaning it causes infinite delay.
- We cannot simply use a DFT or FFT to obtain an ideal filter in the frequency domain either. Because the DFT also represents a filter bank, but a special type.
- Its equivalent filters are far from perfect filters (hence we cannot make ideal filters with it), not good enough for our purposes (audio coding and the ear).

Cosine modulated filter banks

- Cosine modulation of a lowpass prototype filter can realize parallel M-channel filter banks with nearly perfect reconstruction (called “Pseudo QMF”).
- Analysis and synthesis filters satisfy the mirror image condition

$$g_k(n) = h_k(L - 1 - n)$$

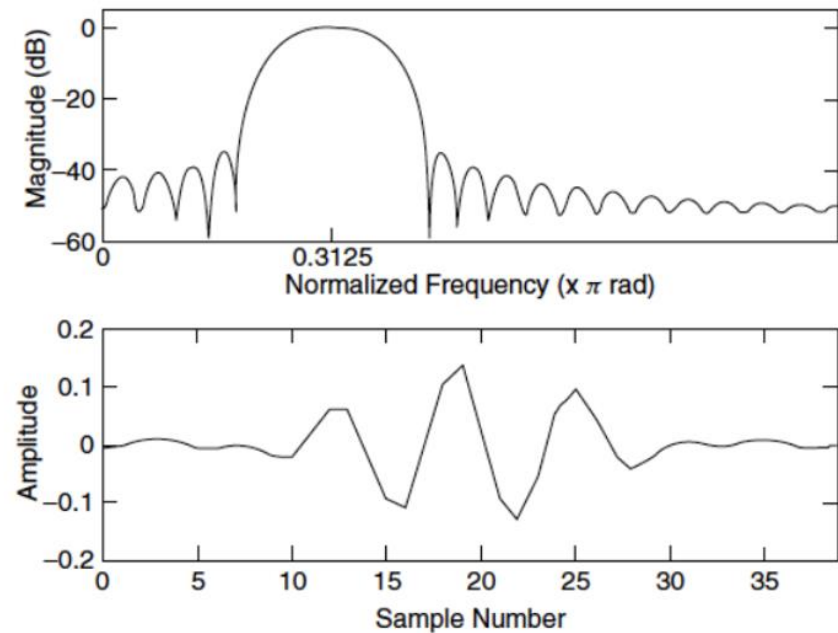
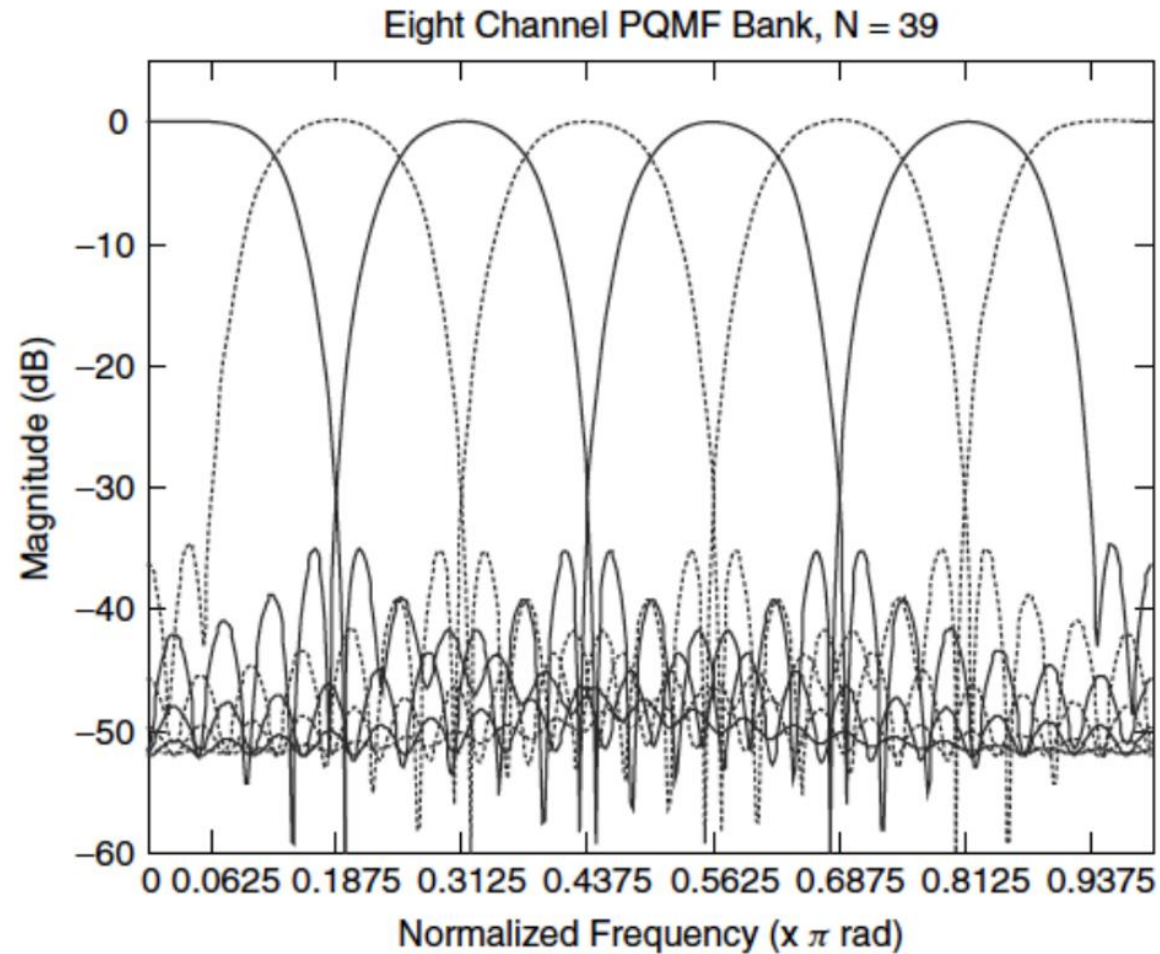
$$h_k(n) = 2w(n) \cos[(\pi/M)(k + 0.5)(n - (L-1)/2) + \Omega_k]$$

$$g_k(n) = 2w(n) \cos[(\pi/M)(k + 0.5)(n - (L-1)/2) - \Omega_k]$$

$$\text{where, } \Omega_k = (-1)^k \pi/4$$

and the sequence $w(n)$ corresponds to the L-sample “window”, a real-coefficient linear phase FIR prototype lowpass filter, with normalized cutoff frequency $\pi/2M$.

Pseudo Quadrature Mirror Filters (PQMF)



Magnitude frequency response and impulse response for channel 3.

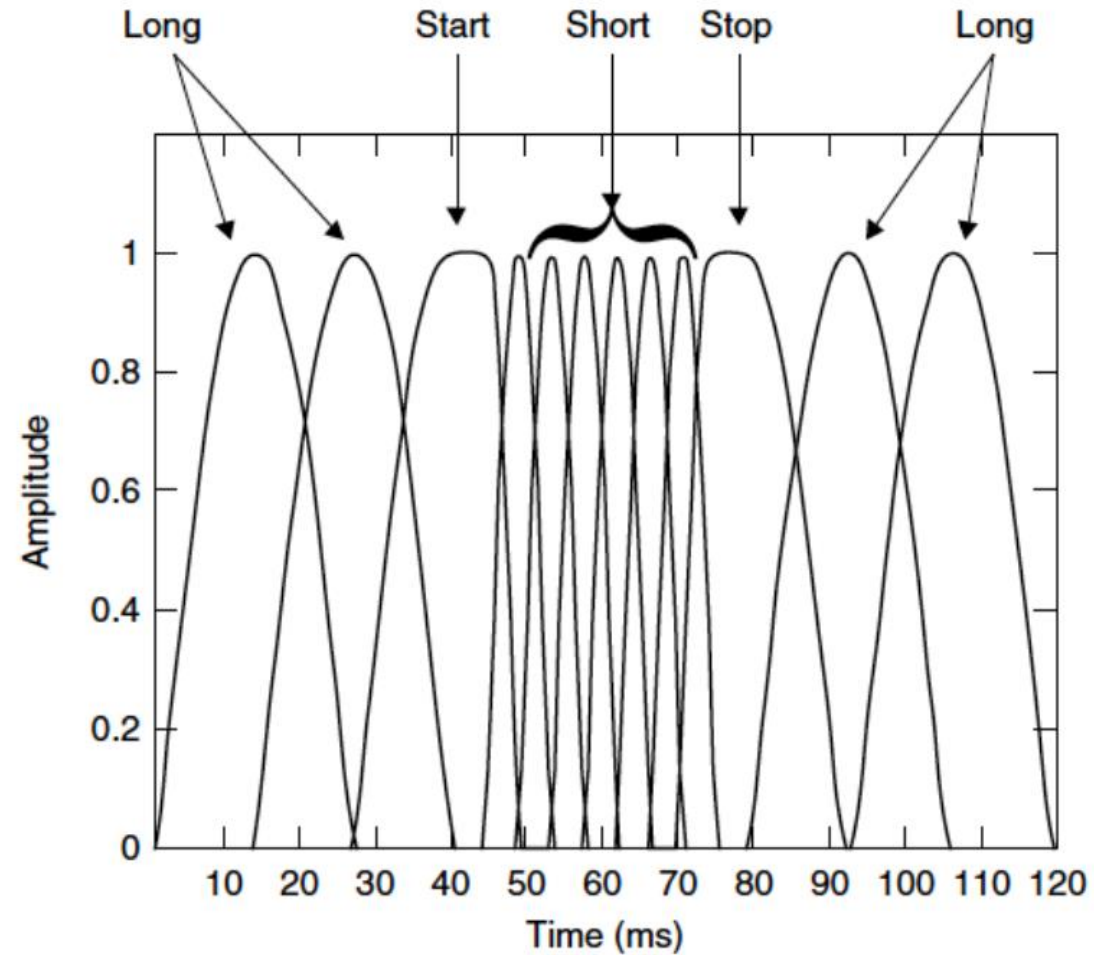
PQMF properties

- Constrained design: single FIR prototype filter
- Uniform, linear phase channel responses
- Overall linear phase, hence constant group delay
- Low complexity, i.e. one filter plus modulation
- Amenable to fast block algorithms
- Critical sampling

Implementation

- Analysis Filter Bank: The input audio signal is passed through a bank of analysis filters, which splits the signal into multiple subbands.
- Downsampling: Each subband is then downsampled to reduce the data rate.
- Encoding/Decoding (Optional): Quantization or other encoding techniques can be applied to each subband for compression.
- Upsampling: The downsampled subbands are upsampled back to their original rate.
- Synthesis Filter Bank: The upsampled subbands are passed through a synthesis filter bank to reconstruct the audio signal.

Hybrid, switched filter banks



DFT, DCT, STFT and MDCT: how they relate

Each representation describes a waveform using weights on basis patterns.

Type	What it does	How it handles time
DFT	One block, complex sinusoidal weights.	Phase describes timing within the block.
DCT-IV	One block, real weights on fixed cosines.	Timing is encoded across the weights.
STFT	A DFT on each windowed frame.	Frame position tracks changes over time.
MDCT	A cosine transform on lapped frames.	Neighbouring frames reconstruct together.

FFT is a fast algorithm for computing the DFT.

Discrete Fourier Transform (DFT)

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1$$

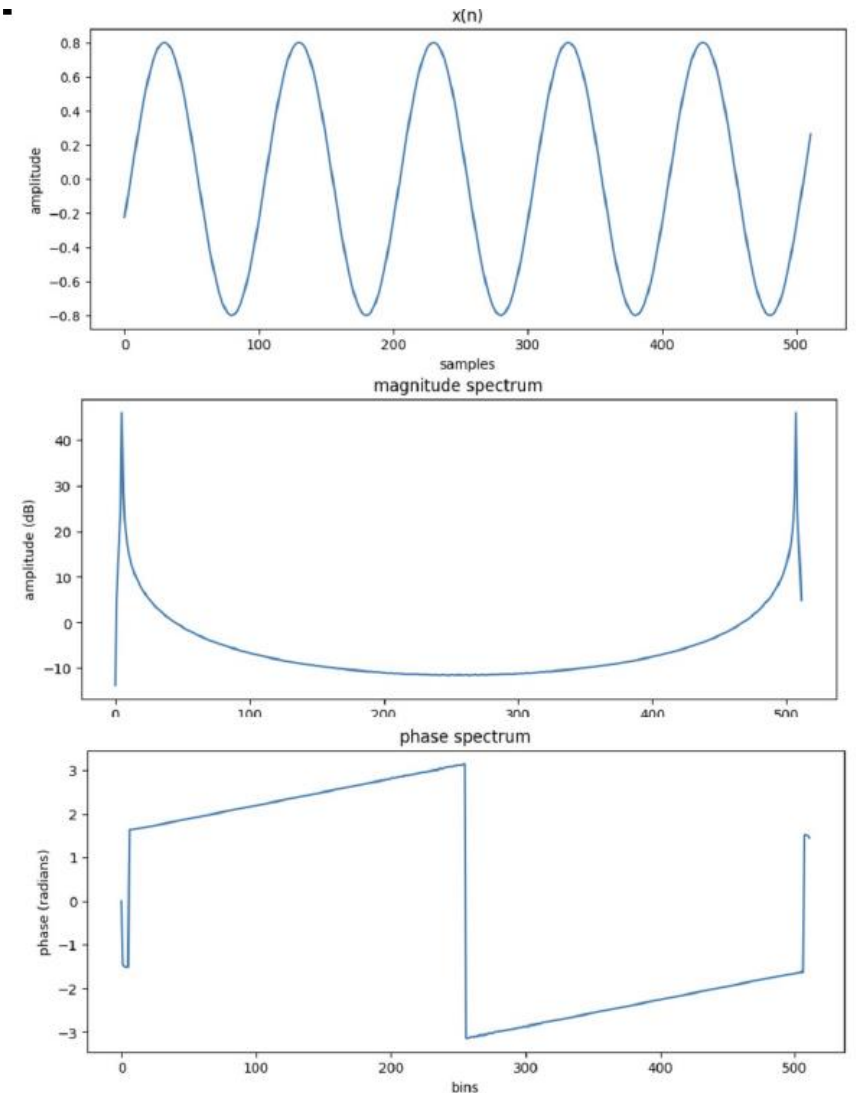
n : sample index k : frequency-bin index

N : samples in this block

$\omega_k = 2\pi k/N$: radians/sample

$f_k = f_s k/N$: Hz (positive-frequency bins)

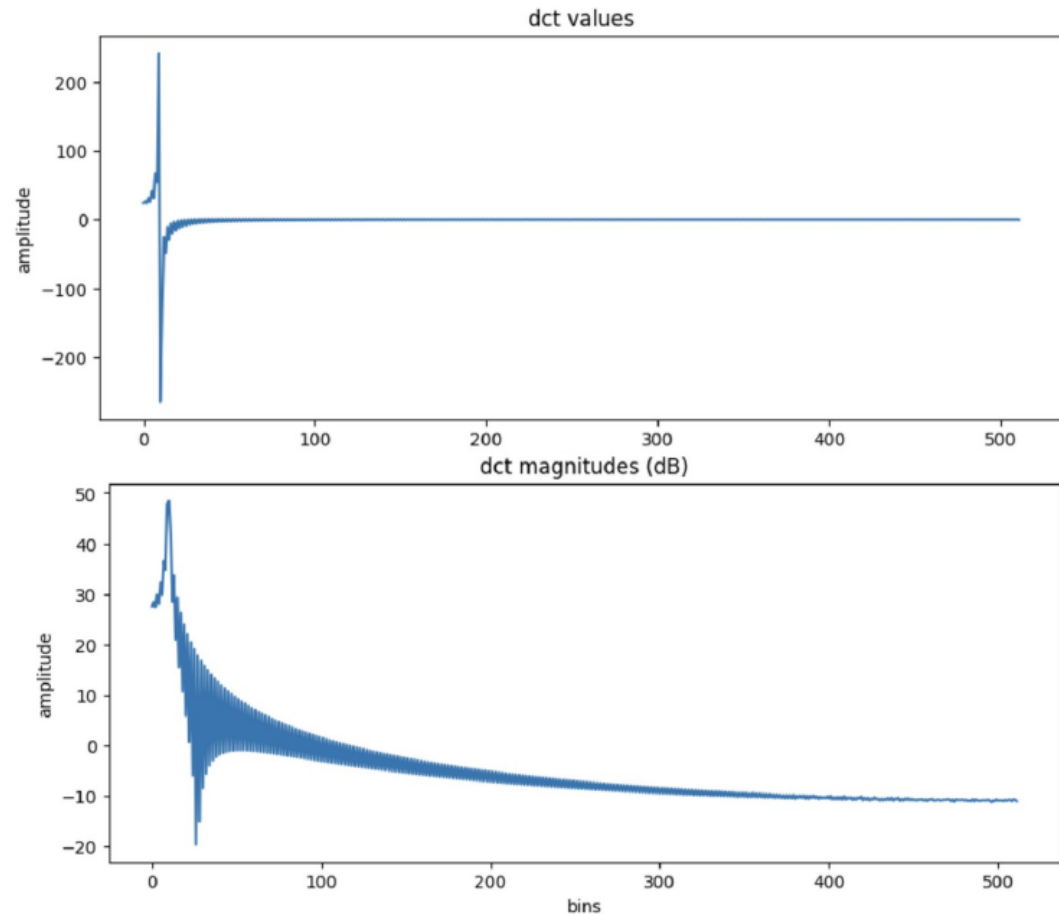
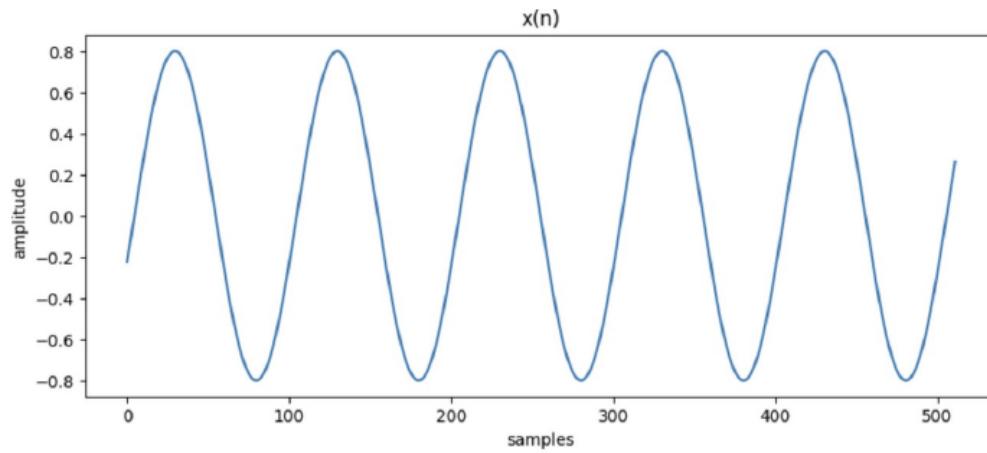
$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}, \quad n = 0, 1, \dots, N-1$$



Discrete Cosine Transform (DCT)

$$y_k(m) = \sqrt{\frac{2}{N}} \sum_{n=0}^{N-1} x(mN + n) \cdot \cos\left[\frac{\pi}{N} (n + 0.5)(k + 0.5)\right]$$

DCT-IV



DCT: cosine weights, phase and compression

A different set of coordinates

- DFT: cosine and sine components give an explicit magnitude and phase.
- DCT: fixed cosine patterns receive real, signed weights.
- Shifting a waveform changes its DCT weights. The weighted cosines preserve its timing.

Why it helps compression

- Correlated samples often give a few large coefficients and many small ones.
- Code important coefficients accurately and quantise others more coarsely.
- Quantisation and coding reduce the bitrate. The transform itself is invertible.

An N-point DCT and the DFT of N real samples both represent N real degrees of freedom.

DCT coefficients preserve timing

$y_k(m)$ is the weight of cosine k in frame m .

A positive weight adds a cosine. A negative weight adds its sign-reversed version.

Example: orthonormal DCT-IV, $N = 4$. Delay a pulse by one sample.

k	Pulse [1, 0, 0, 0]	Delayed pulse [0, 1, 0, 0]
0	+0.694	+0.588
1	+0.588	−0.138
2	+0.393	−0.694
3	+0.138	−0.393

The delay changes several weights. Inverse DCT recovers each pulse.

Short-time Fourier Transform (STFT)

$$X_l[k] = \sum_{n=-\frac{N}{2}}^{\frac{N}{2}-1} w[n]x[n + lH]e^{-j2\pi kn/N}, \quad l = 0, 1, \dots$$

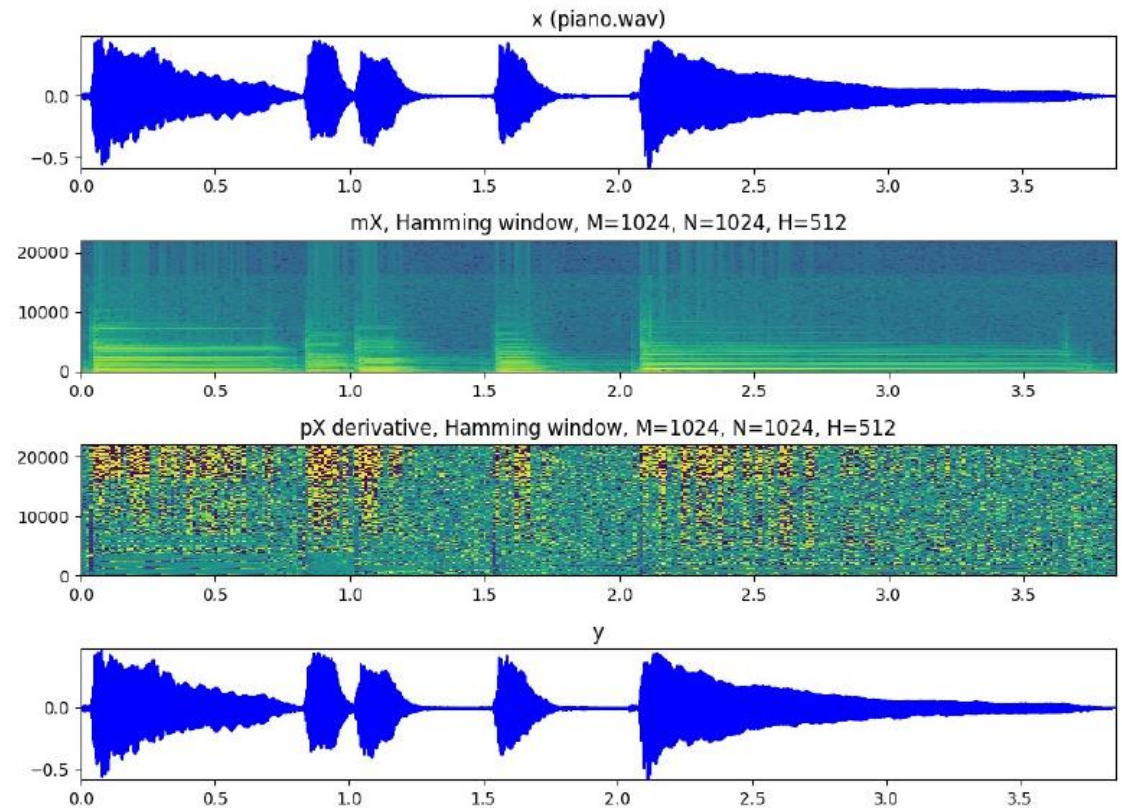
w : analysis window

l : frame number

H : hop-size

$$y[n] = \sum_{l=0}^{L-1} \text{Shift}_{lH,n} \left[\frac{1}{N} \sum_{k=-\frac{N}{2}}^{\frac{N}{2}-1} X_l[k] e^{j2\pi kn/N} \right]$$

Synthesis windows and gain normalization are implicit here.



STFT: window, hop and reconstruction

N samples per frame, H samples between frames, w is the analysis window.

At 48 kHz: N = 1,024 gives a 21.3 ms window. H = 256 gives a 5.3 ms hop.

A smaller hop gives more frames. Frequency resolution still depends on the window.

Let $v_l[r]$ be the inverse DFT of frame l. It recovers the windowed samples.

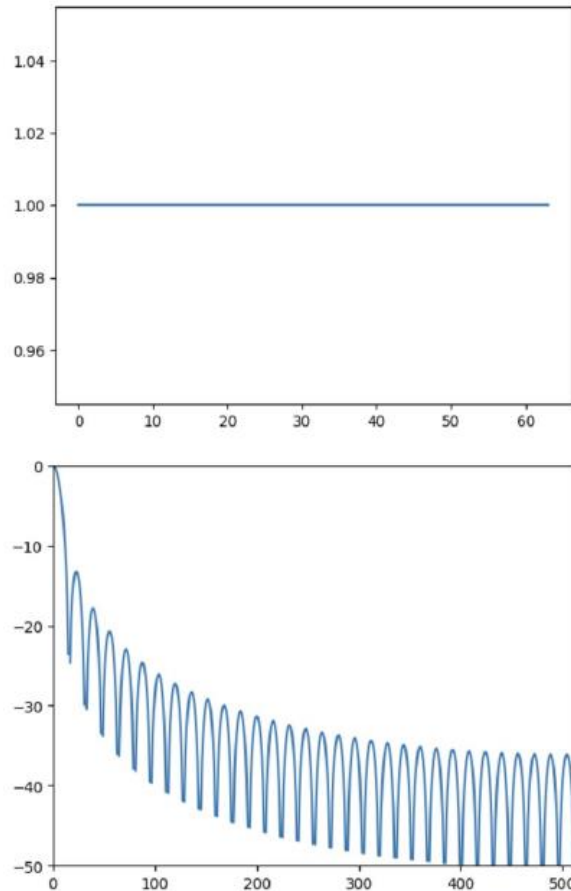
$$x[n] = \frac{\sum_{l=0}^{L-1} w[n - lH] v_l[n - lH]}{\sum_{l=0}^{L-1} w^2[n - lH]}$$

A magnitude spectrogram hides phase. The complex STFT retains it.

Windows

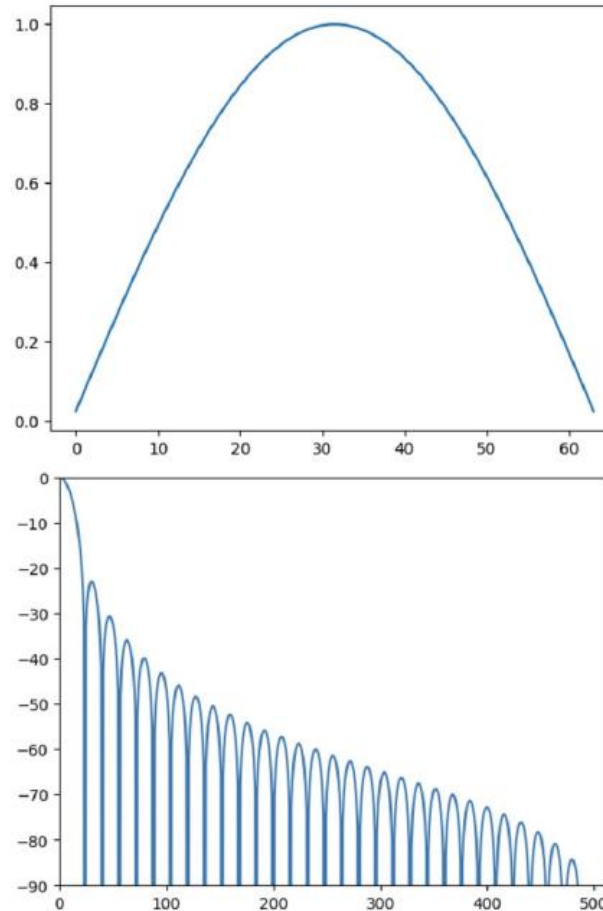
Rectangular window:

$$w[n] = 1$$



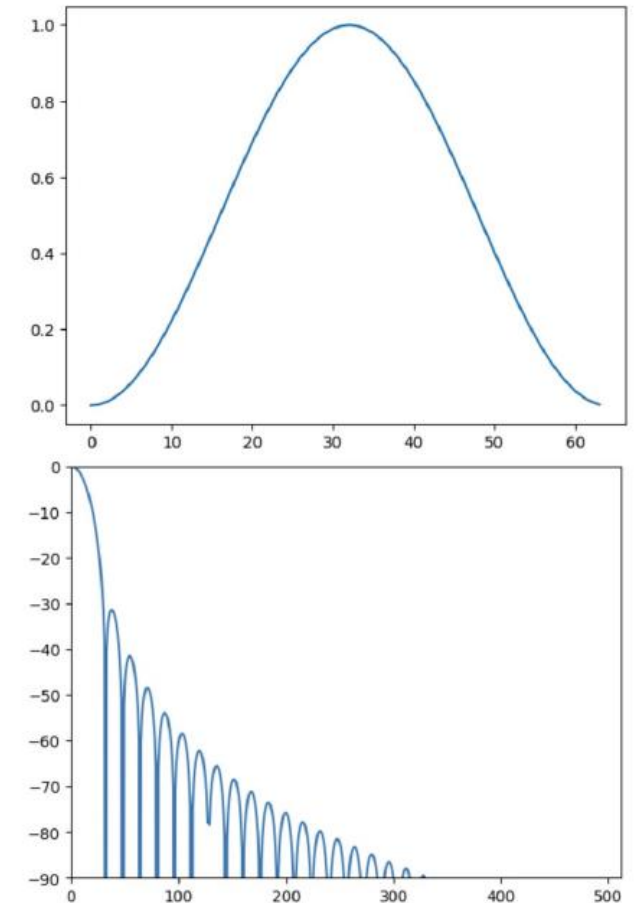
Sine window:

$$w(n) = \sin[(n + 1/2) \pi / (2M)]$$



Hanning window:

$$w[n] = .5 + .5 \cos(2\pi n / M)$$



Modified Discrete Cosine Transform (MDCT)

- Uses 50% window overlap while remaining critically sampled.
- Transforms $2N$ input samples into N real coefficients, with an N -sample hop.
- Inverse frames contain time-domain aliasing. Suitable windows and overlap-add cancel it for perfect reconstruction before quantisation.
- Efficient implementations use DCT-IV or FFT algorithms.

$$h_k(n) = h(n) \sqrt{\frac{2}{N}} \cos[(\pi/N)(k + 1/2)(n + N/2 + 1/2)]$$

$h_k(n)$ is the windowed cosine pattern for channel k .

Its MDCT coefficient sums $x[n] \cdot h_k(n)$ over the $2N$ samples.

MDCT: overlap without redundant coefficients

Frame accounting

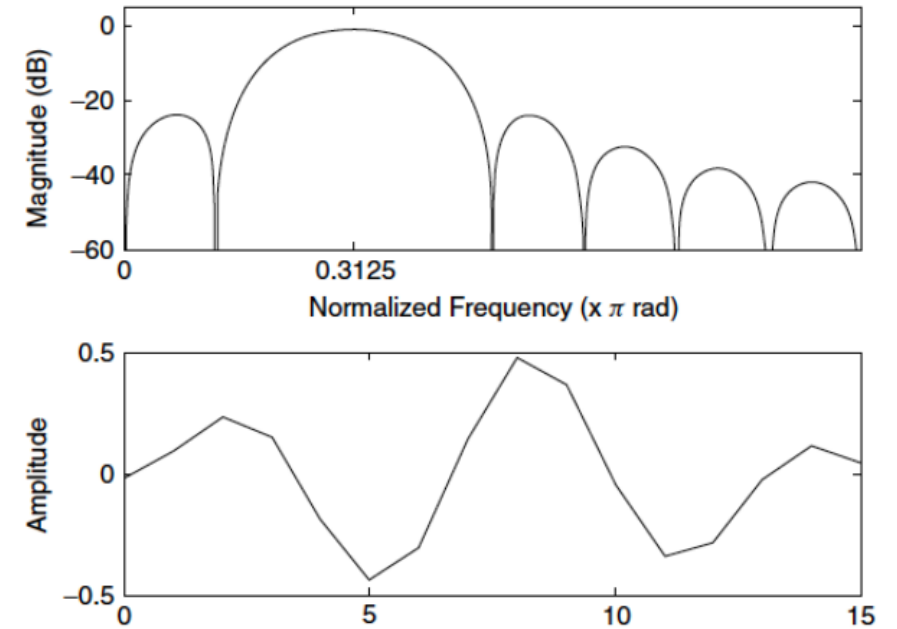
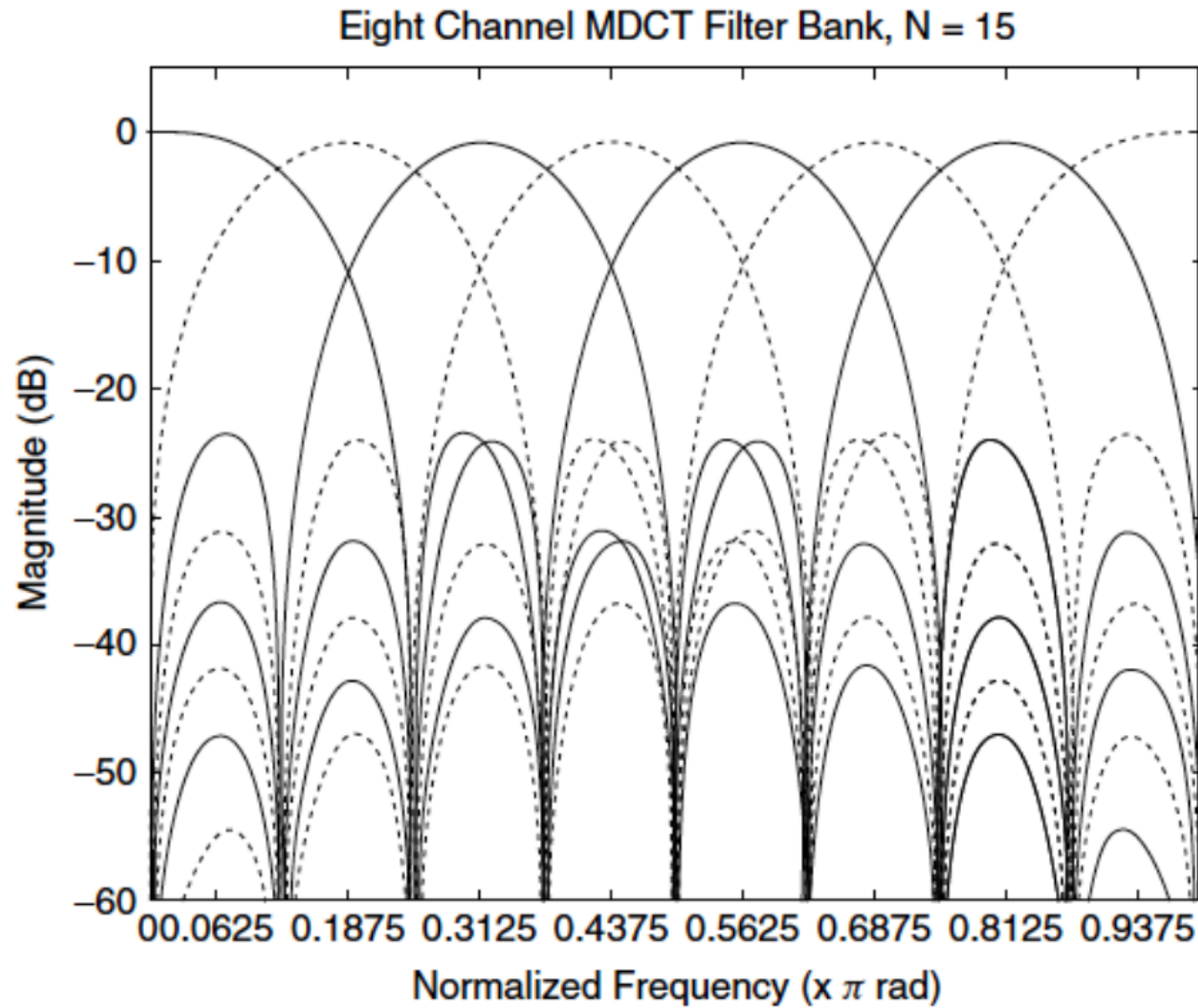
2N input samples per block
N-sample hop (50% overlap)
N real coefficients per block

Example: 1,024 samples per block
512-sample hop, 512 coefficients

Reconstruction across frames

- A single inverse frame contains deliberate time-domain aliasing.
- Overlapping inverse frames cancel that aliasing when added with suitable windows.
- The complete stream reconstructs exactly before quantisation.

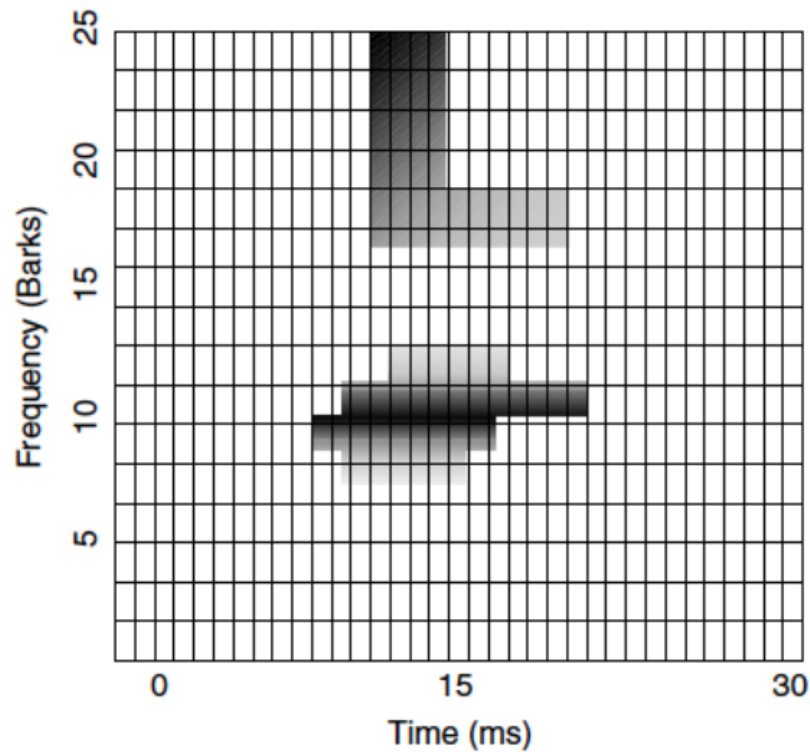
Smooth frame transitions with one real coefficient per new input sample.



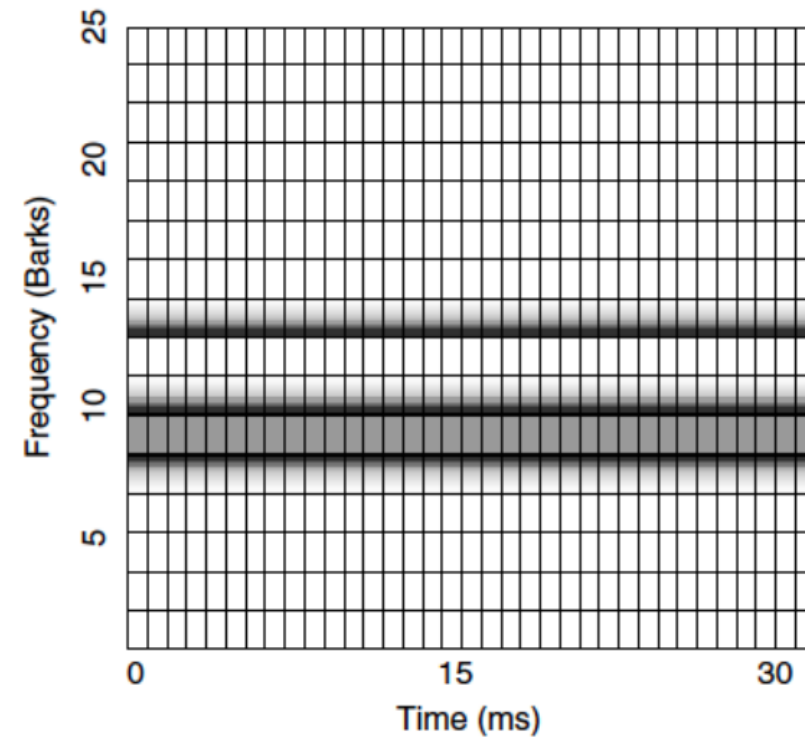
Magnitude frequency response and impulse response for channel 3.

Time-frequency resolution issues

Difficulty of selection a single filter bank to satisfy the diverse time and frequency resolutions requirements associated with different classes of audio.



castanets

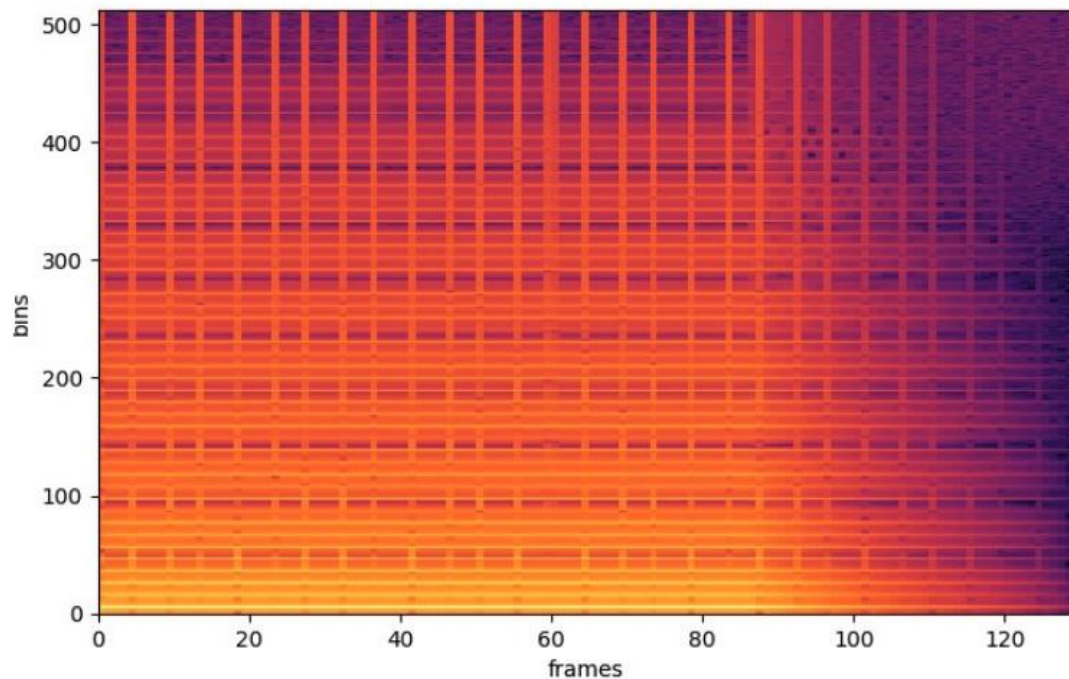


piccolo

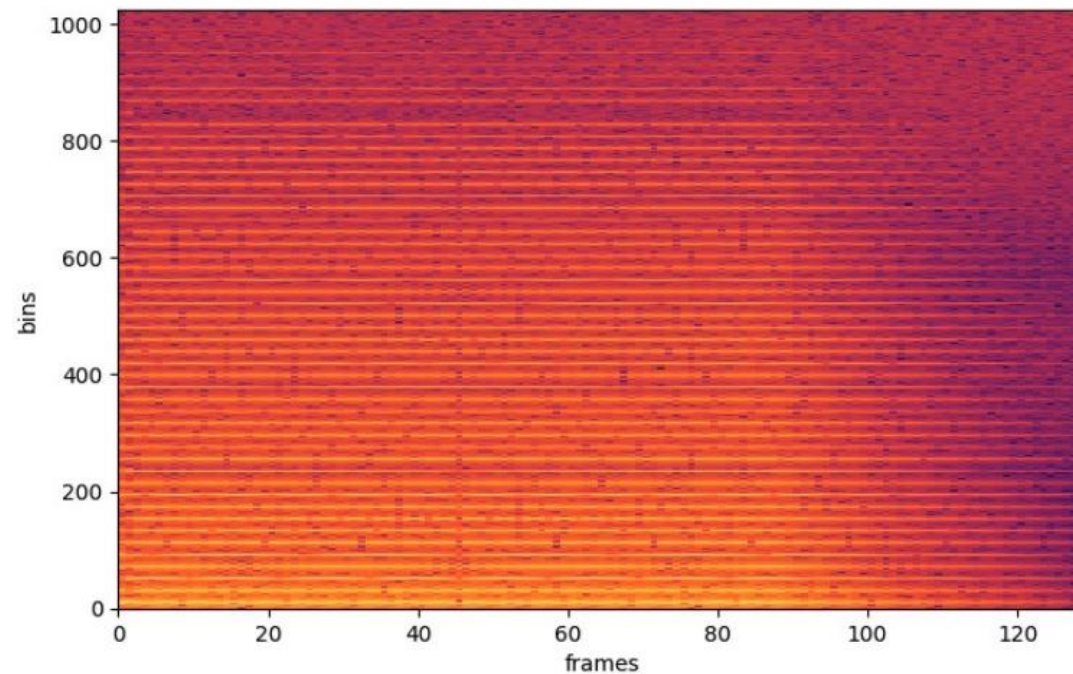
STFT vs MDCT

- STFT can use overlapping windows. The left example uses none.
- MDCT uses 50% window overlap and remains critically sampled.

STFT, no window overlap



MDCT



STFT and MDCT: representation and reconstruction

Same $2N$ -sample block and N -sample hop (50% overlap), with real input.

	STFT	MDCT
Coefficients	Complex values give explicit magnitude and phase.	Real cosine weights encode timing across overlapping frames.
Coefficient rate	$2N$ independent real values per frame.	N real coefficients per frame: critically sampled.
Reconstruction	Inverse DFT, compatible windows and overlap-add.	Inverse MDCT, windows and time-domain aliasing cancellation.
Main advantage	Flexible spectral analysis and processing.	Smooth transitions and no overlap redundancy for audio coding.

Key takeaways

- Filter banks split audio into subbands. Reconstruction requires compatible analysis and synthesis.
- DFT describes a block with complex coefficients. DCT describes it with real cosine weights.
- Both preserve waveform timing. DCT represents it across the signed coefficients.
- STFT follows spectral changes with successive windowed DFTs and can use overlap.
- MDCT combines overlap with critical sampling. Adjacent inverse frames cancel aliasing.
- Quantisation and coding reduce bitrate. The transform alone does not compress the data.
- Window length controls time and frequency resolution. Hop size controls frame spacing.

References and notebooks

Core reading

Andreas Spanias, Ted Painter, Venkatraman Atti (2007). Audio Signal Processing and Coding. Chapter 6: Time-frequency analysis: filter banks and transforms.

Course notebooks

- `examples/FIR-design.ipynb`
- `examples/filter-bank-basic.ipynb`
- `examples/filter-bank-critically-sampled.ipynb`
- `examples/dft-dct.ipynb`
- `examples/dct-mcdt.ipynb`
- `examples/windows.ipynb`
- `examples/stft-mdct.ipynb`

github.com/xserra/audio-coding-materials